



HeeraMon

Data Model / Database Specification

Data Model Specification Document

Date of Preparation: September 30, 2026

Document ID	HM-DOC-08
Title	Data Model / Database Specification
Classification	Data Model Specification Document
Status	Draft
Version	1.0
Owner	Product / Engineering
Audience	All project contributors
Related Documents	Technical Architecture (HM-DOC-07), SRS (HM-DOC-02)

Contents

1	Purpose	2
2	Design Principles	2
3	Core Entities	2
3.1	User	2
3.2	User Preference	3
3.3	Conversation	3
3.4	Message	3
3.5	Job	4
3.6	Job Source	4
3.7	Job Source Record	4
3.8	Match	5
3.9	User Decision	5
3.10	Application	6
3.11	Notification	6
3.12	Agent Run / Task	6
4	Core Relationships	7
5	Entity-Relationship Diagram	8
6	Important Data Rules	8
6.1	Preference History	8
6.2	Job Deduplication	8
6.3	User Isolation	8
6.4	Application State	8
6.5	Agent State	9
6.6	Unknown Values	9
7	Indexing and Performance	9
8	Boundaries	9
9	Success Criteria	9
10	Document Control	10

1 Purpose

This document defines the persistent data model for HeeraMon.

The database should provide a reliable source of truth for:

- users;
- preferences;
- conversations;
- jobs;
- matches;
- decisions;
- applications;
- notifications;
- ingestion sources;
- agent/background state.

PostgreSQL is the proposed database.

2 Design Principles

The data model should:

- keep persistent application state separate from temporary LLM output;
- preserve user decisions and preference history;
- distinguish explicit preferences from learned preferences;
- preserve job-source information;
- support background processing;
- support auditability of consequential actions;
- avoid unnecessary duplication;
- remain extensible for future versions.

3 Core Entities

3.1 User

Represents an authenticated HeeraMon user.

Key information:

- id
- authentication/provider identifiers
- name

- contact information
- profile information
- created/updated timestamps

3.2 User Preference

Represents a requirement or preference extracted from conversation.

Key information:

- id
- user_id
- category
- value
- type: hard / soft / learned
- source: explicit / confirmed / inferred
- confidence
- status
- created/updated timestamps

Preference history should be preserved where changes affect future matching.

3.3 Conversation

Represents a user-agent conversation or conversation thread.

Key information:

- id
- user_id
- title/context metadata
- created/updated timestamps

3.4 Message

Represents an individual conversational message.

Key information:

- id
- conversation_id
- role
- content
- metadata
- timestamp

3.5 Job

Represents a normalized job opportunity.

Key information:

- id
- title
- company
- description
- location
- salary
- work arrangement
- employment type
- requirements
- benefits
- application URL
- status
- first-seen/last-seen timestamps

3.6 Job Source

Represents where a job originated.

Key information:

- id
- source name
- source type
- source URL/configuration
- status
- last successful sync
- error information

3.7 Job Source Record

Represents the relationship between a normalized job and its original source record.

This allows HeeraMon to preserve source-specific identifiers and URLs while maintaining one normalized job.

Key information:

- id
- job_id

- source_id
- external job ID
- source URL
- source metadata
- first-seen/last-seen timestamps

3.8 Match

Represents HeeraMon's evaluation of a job for a specific user.

Key information:

- id
- user_id
- job_id
- eligibility status
- match information
- satisfied preferences
- violated preferences
- unknown information
- trade-offs
- explanation
- created/updated timestamps

A match belongs to both a user and a job.

3.9 User Decision

Represents a user's response to an opportunity.

Examples:

- interested
- rejected
- saved
- proceed to application
- ignored

Key information:

- id
- user_id
- job_id

- decision
- reason/context where available
- timestamp

These decisions provide evidence for future preference learning.

3.10 Application

Represents an application workflow for a specific user and job.

Key information:

- id
- user_id
- job_id
- status
- prepared application data
- submission status
- external application identifier where available
- timestamps

Possible statuses include:

draft → review → approved → submitted

with appropriate failure/cancellation states.

3.11 Notification

Represents a notification generated for a user.

Key information:

- id
- user_id
- notification type
- related entity
- message/content
- read status
- delivery status
- timestamp

3.12 Agent Run / Task

Represents a background or significant agent execution.

Useful for tracking:

- task type;
- status;
- input/context references;
- execution timestamps;
- errors;
- retry information.

This supports observability and resumable background workflows.

4 Core Relationships

The main relationships are:

- One User has many Preferences.
- One User has many Conversations.
- One Conversation has many Messages.
- One User can have many Matches.
- One Job can have many Matches across different users.
- One User can make many Decisions.
- One Job can have many Decisions from different users.
- One User can have many Applications.
- One Job can have applications from multiple users.
- One Job Source can contain many source records.
- One Job can have records from multiple sources.
- One User can receive many Notifications.
- One User can have many Agent Runs / Tasks.

5 Entity-Relationship Diagram

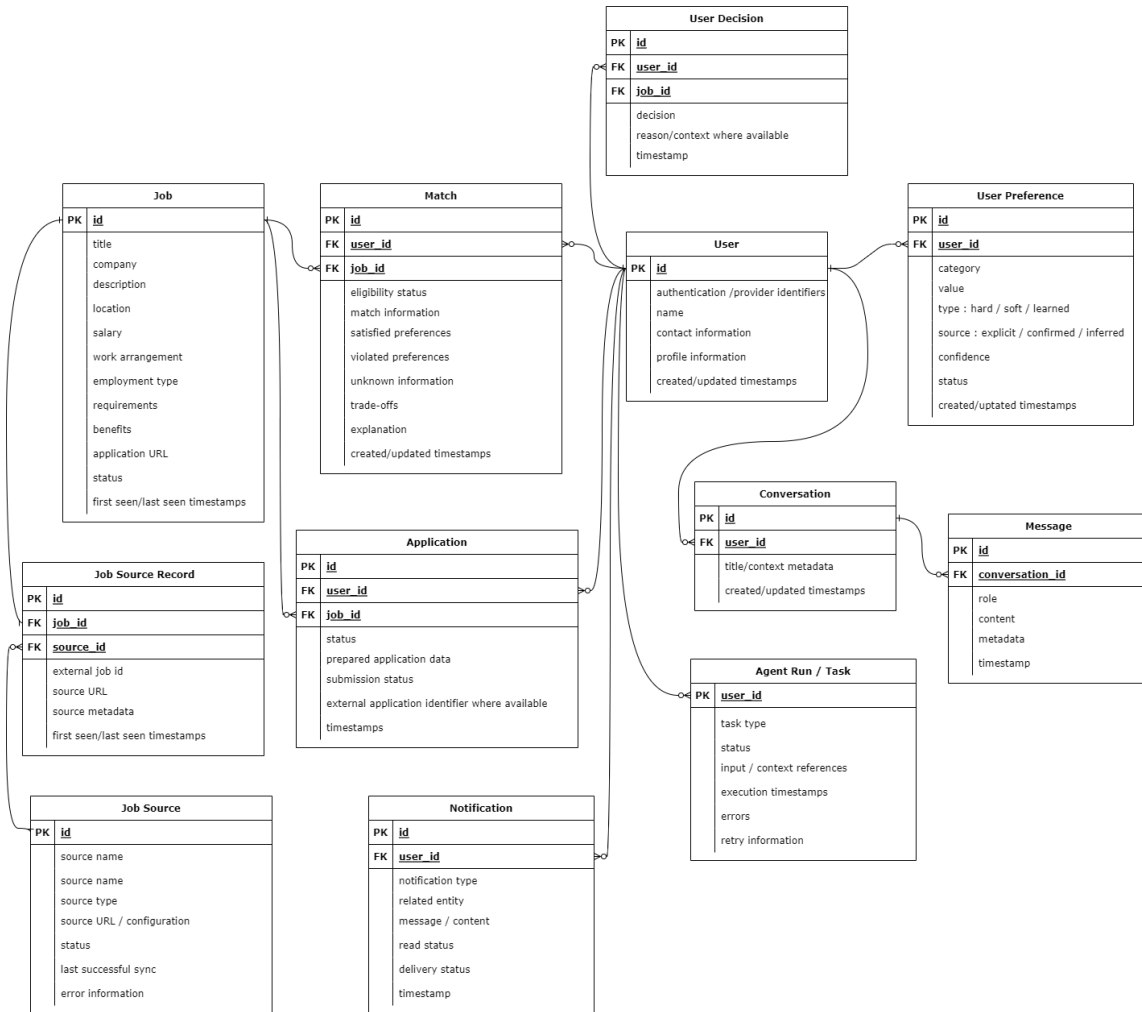


Figure 1: Core HeeraMon entity-relationship model.

6 Important Data Rules

6.1 Preference History

Preference changes should not silently overwrite important historical context.

6.2 Job Deduplication

Multiple source records may point to the same normalized job.

6.3 User Isolation

A user's private data must only be accessible within their authorized account context.

6.4 Application State

Application status must be controlled by backend logic and explicit user actions, not by arbitrary LLM output.

6.5 Agent State

LLM-generated state should not become authoritative unless validated and persisted by application logic.

6.6 Unknown Values

Unknown job or preference information should remain explicitly unknown rather than represented as guessed values.

7 Indexing and Performance

The system should index frequently queried fields such as:

- user IDs;
- job status;
- source IDs;
- external job IDs;
- job URLs;
- preference ownership;
- match ownership;
- application status;
- timestamps used for background processing.

Indexes should be added based on actual query patterns rather than indiscriminately.

8 Boundaries

The initial release does not require:

- a separate data warehouse;
- complex analytics schemas;
- a dedicated vector database;
- distributed databases;
- large-scale event sourcing;
- advanced recommendation-specific storage.

The relational PostgreSQL model should remain the primary source of truth.

9 Success Criteria

The data model is successful when it can reliably represent:

1. A user's identity and profile.

2. Their explicit and learned preferences.
3. Their conversations and decisions.
4. Normalized jobs and their sources.
5. User-specific job matches.
6. Application preparation and submission state.
7. Notifications.
8. Background agent activity.

Core principle: The database should preserve the history and state needed to make HeeraMon continuous, explainable, and trustworthy.

10 Document Control

Version	Date	Author / Role	Changes
1.0	September 30, 2026	Product / Engineering	Initial release

Approval: This document defines the persistent data model for HeeraMon. All subsequent design and implementation work must remain consistent with the principles stated herein.

HeeraMon flies to help people find job that fits not only their career, but the life they want to live.



Saadman Sakib
Product Engineer